

Tracing the Markowitz Efficient Frontier with OSQP: a Short Tutorial on the OR-Library Portfolio Benchmarks

Heuristic Algorithms in Transport and Finance — Master's course assignment

Javier Salvatierra

Jesús Inurria

September 2026

Abstract

The basic Markowitz portfolio selection problem is a convex quadratic program (QP): minimise the variance of the portfolio return subject to a minimum expected return, a budget constraint and no short selling. We formulate it, map it to the standard form of OSQP — an open-source first-order (ADMM) solver — and trace the unconstrained efficient frontier of the five OR-Library instances `port1–port5` by solving the 2000 QPs given by the return levels of `portef1–portef5`. The frontiers match the reference ones to the rounding of the benchmark files ($\sim 10^{-5}\%$ in variance), at 0.4–137 ms per QP. Near the maximum return the problem becomes degenerate and 63 of the 10 000 points are not solved to tolerance, which illustrates the practical limits of a first-order method.

1 Introduction

An investor has a fixed budget and N assets to spread it over. More expected return normally means more uncertainty, so there is no single best portfolio but a trade-off. Markowitz made that trade-off computable by measuring risk as the *variance* of the portfolio return, which turns portfolio selection into an optimisation problem with a quadratic objective and linear constraints. Beasley's tutorial [1] presents this basic model together with the extensions that make it hard in practice (cardinality limits, transaction costs, index tracking); we deal only with the basic version, without short selling.

The test data are the five portfolio instances of OR-Library [2]. It is built from weekly prices of real stocks between March 1992 and September 1997: Hang Seng, DAX 100, FTSE 100, S&P 100 and Nikkei 225, with N between 31 and 225 (see Table 2). Each file `portk.txt` stores the mean return and the standard deviation of every asset plus all pairwise correlations; the companion file `portefk.txt` stores 2000 points (mean return, variance) of the corresponding *unconstrained efficient frontier* (UEF), which we use as the reference solution. The rest of the paper (i) states the model, (ii) explains why we chose OSQP, and (iii) reports times and accuracy on the five instances.

2 The quadratic programming model

2.1 Parameters and decision variables

For each asset $i = 1, \dots, N$ the data give the expected (mean) return μ_i and the standard deviation s_i of its return, and for each pair the correlation $\rho_{ij} \in [-1, 1]$. The decision variable w_i is the proportion of the budget invested in asset i , and $R_{\min}$ is the return required from the portfolio. All statistics are weekly: $\mu_i = 0.0109$ means 1.09 % per week.

2.2 Formulation

$$\min_{w_1, \dots, w_N} \sum_{i=1}^N \sum_{j=1}^N s_i s_j \rho_{ij} w_i w_j \quad \text{s.t.} \quad \sum_{i=1}^N \mu_i w_i = R_{\min}, \quad \sum_{i=1}^N w_i = 1, \quad w_i \geq 0. \quad (1)$$

Read literally: the objective is the variance of the portfolio return, obtained by adding, over every pair of assets, the product of the two risks, their correlation and the two amounts invested. The first constraint says that the portfolio must earn at least $R_{\min}$ on average. The second says that the whole budget is invested. The third forbids short selling, which is caused by investing negative quantity any asset. The upper bound $w_i \leq 1$ need not be written, since it follows from $w_i \geq 0$ and $\sum_i w_i = 1$.

Writing $\sigma_{ij} = s_i \rho_{ij} s_j$ for the covariance and $\Sigma = [\sigma_{ij}]$ for the covariance matrix, the objective becomes the compact quadratic form $w^\top \Sigma w$. Beasley writes the return constraint as an equality [1]; both versions give the same portfolio for every $R_{\min}$ used here, because the constraint is active at the optimum (see below).

2.3 Convexity

A covariance matrix is always positive semidefinite, and in the five instances it is in fact positive definite: the smallest eigenvalue ranges from $6.1 \cdot 10^{-6}$ (`port5`) to $2.3 \cdot 10^{-4}$ (`port1`). Problem (1) is therefore a strictly convex QP: every local minimum is global, the optimal portfolio is *unique*, and the KKT conditions

$$2\Sigma w - \lambda \mu - \nu \mathbf{1} - s = 0, \quad \lambda, s \geq 0, \quad \lambda (\mu^\top w - R_{\min}) = 0, \quad s_i w_i = 0$$

are necessary and sufficient. This is what makes the model easy in the sense of Beasley: the objective is non-linear, but efficient algorithms exist and the global optimum is found without difficulty. The multiplier λ is the shadow price of return, i.e. the slope dV/dR of the frontier.

2.4 The efficient frontier

Solving (1) for one value of $R_{\min}$ gives one portfolio; sweeping $R_{\min}$ traces the efficient frontier, the set of Pareto-optimal (non-dominated) portfolios. Its lower end is the global minimum-variance portfolio (GVM), obtained by dropping the return constraint. The upper end is $R_{\max} = \max_i \mu_i$, attained only by putting the whole budget in the single asset with the highest mean return. The minimum variance $V^*(R)$ is convex, non-decreasing and piecewise quadratic, each piece corresponding to a fixed set of assets held. Individual assets, and any portfolio off the curve, are inefficient: they lie below and to the right of it (red points in Figure 1).

3 Choice of solver

Table 1 lists the options suggested in the assignment. We chose OSQP [3], an open-source solver written in C that applies ADMM [3] to problems in the standard form

$$\min_w \frac{1}{2} w^\top P w + q^\top w \quad \text{s.t.} \quad l \leq A w \leq u, \quad P \in \mathbb{S}_+^N. \quad (2)$$

Model (1) maps onto (2) by taking $P = 2\Sigma$ (the factor 2 cancels the $\frac{1}{2}$, so that the objective value returned is exactly the variance), $q = 0$, and one row of A per constraint: row 0 is $\mu^\top$ with $l_0 = R_{\min}$ and $u_0 = +\infty$, row 1 is $\mathbf{1}^\top$ with $l_1 = u_1 = 1$, and the remaining N rows are the identity with $l = 0$, $u = +\infty$.

Table 1: Candidate solvers for a convex QP.

Solver	Algorithm for convex QP	Licence	Comment for this problem
Gurobi / CPLEX	Barrier (interior point), simplex	Commercial (free academic)	Robust and accurate; also MIQP; need a licence
HiGHS	Active set	MIT	Accurate and lightweight; no MIQP
SCIP	Branch-and-bound for MINLP	Apache 2.0	Oversized for a convex QP; useful with cardinality
OSQP	ADMM / operator splitting	Apache 2.0	Chosen

The decisive argument is the *structure of the task*: a frontier is not one QP but 2000 almost identical QPs in which only the scalar $l_0 = R_{\min}$ changes. Furthermore, it is used as a python library which makes the implementation easier.

- **One factorisation for the whole frontier.** The bounds l, u do not enter the KKT matrix, so setup factorises it once (5–12 ms here) and update ($l=...$) changes the required return without refactorising.
- **Warm start.** Each solve restarts from the previous portfolio, which is very close to the next one.
- **Accuracy settings.** The user controls the tolerance, the iteration limit and *solution polishing*, a final linear solve on the detected active set that lifts an ADMM solution to near machine precision. Iterations are cheap, there is no licence and installation is `pip install osqp`.

The price to pay is that ADMM is a first-order method: it needs tuning (with the default tolerance 10^{-3} and no polishing we measured variance errors of up to 3.1 %, 11.2 % and 6.7 % on port1–port3) and it converges slowly on degenerate problems (Section 5).

4 Implementation

The code is a Python notebook (`markowitz_osqp.ipynb`, delivered separately) using Python 3.14, NumPy, SciPy, pandas, matplotlib and OSQP 1.1.3. It reads `portk.txt`, builds $\Sigma = \rho \odot ss^\top$, sets the problem up once per instance and then loops over the return levels:

```
for R in R_levels:    l[0] = R;    solver.update(l=l);    res = solver.solve()
```

The settings are `eps_abs = eps_rel = 1e-6`, `polishing = True` and `max_iter = 20000` (OSQP’s default of 4000 is not enough near $R_{\max}$); the rest are left at their defaults. As grid we use the **2000 return levels of portefk** themselves, which allows a point-by-point comparison without interpolation. We record the wall-clock time per instance (setup plus 2000 solves), the mean number of ADMM iterations, the points whose status is not solved, and the relative deviation $100(V_{\text{OSQP}} - V_{\text{portef}})/V_{\text{portef}}$ over the solved points. All runs are single-threaded on an AMD Ryzen 5 5500U laptop under Windows 11. As a sanity check, port1 with $R_{\min} = 0.008$ is solved in 125 iterations and 1.0 ms; the portfolio holds 4 of the 31 assets (40.1, 37.5, 16.7 and 5.7 %), returns exactly 0.008000 and has variance $1.545024 \cdot 10^{-3}$.

5 Computational results

Table 2 collects the results and Figure 1 shows the five frontiers.

Table 2: Tracing the five frontiers: 2000 QPs per instance, OSQP 1.1.3, single thread.

Instance	Index	N	Time (s)	ms/QP	Mean iter.	Not solved	Max dev (%)	Mean dev (%)
port1	Hang Seng	31	0.87	0.44	76	0	$7.8 \cdot 10^{-6}$	$2.9 \cdot 10^{-6}$
port2	DAX 100	85	25.93	12.96	797	0	$3.6 \cdot 10^{-5}$	$1.0 \cdot 10^{-5}$
port3	FTSE 100	89	10.08	5.04	255	0	$3.8 \cdot 10^{-2}$	$3.6 \cdot 10^{-5}$
port4	S&P 100	98	33.90	16.95	803	3	$1.0 \cdot 10^{-2}$	$1.7 \cdot 10^{-5}$
port5	Nikkei 225	225	274.96	137.48	974	60	$1.1 \cdot 10^{-1}$	$1.2 \cdot 10^{-4}$

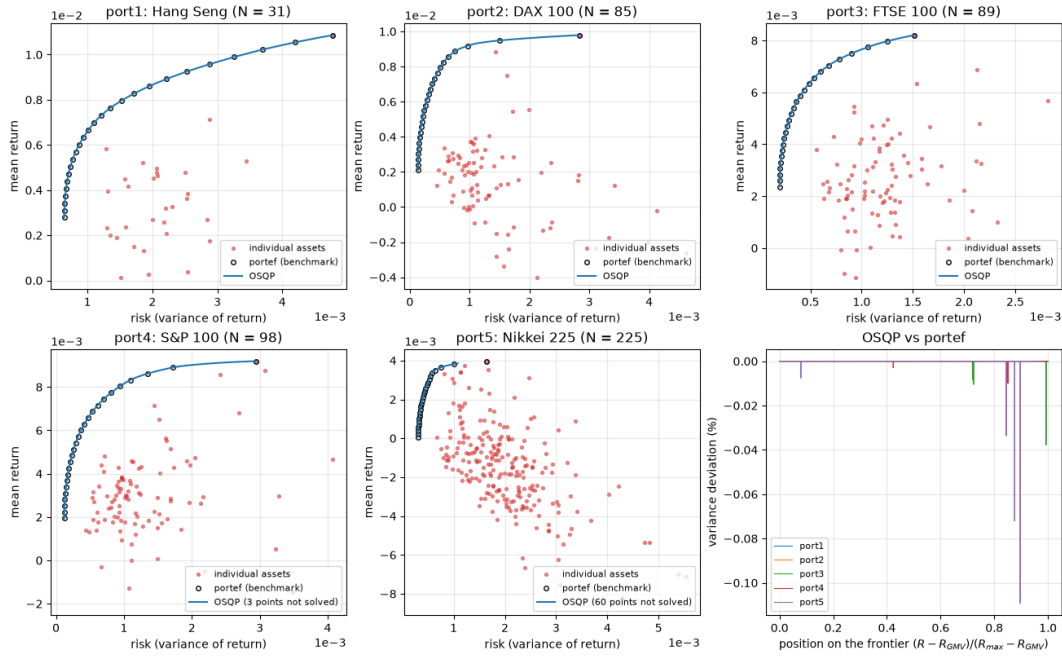


Figure 1: Panels 1–5: unconstrained efficient frontier computed with OSQP (blue line), 26 reference points of *portef* (black circles) and the individual assets (red dots), which are inefficient portfolios. Panel 6: relative deviation in variance along the frontier, with position measured as $(R - R_{\text{GMV}})/(R_{\text{max}} - R_{\text{GMV}})$.

Accuracy. Over almost the whole curve the agreement with the benchmark is of order $10^{-5}\%$, which is simply the rounding of *portef* (10 decimals for variances of order 10^{-4}): It reproduces the reference frontiers exactly to the precision at which they are published. The isolated spikes of up to 0.11 % in panel 6 are a handful of points where polishing failed; the solution then stays at ADMM accuracy, violates the return constraint by $\sim 10^{-8}$ and therefore reports a slightly *lower* variance.

Computational time. Tracing a whole frontier costs from 0.87 s (port1) to about 4.5 min (port5), roughly 5.8 min for the 10 000 QPs, i.e. 0.44–137 ms per QP. Time is not a function of N alone — port3 ($N = 89$) averages 255 iterations while port2 ($N = 85$) needs 797 — but of conditioning and of the geometry of the upper end. Profiling shows that $\sim 96\%$ of the time is spent inside OSQP’s C iterations and that the last 5 % of the return levels consume 73 % of the time of port5; timings fluctuate between runs (183–275 s).

The failures at the top end. 63 of the 10 000 points are not solved to tolerance: 3 in port4 ($R \in [0.009090, 0.009097]$, $R_{\text{max}} = 0.009195$) and 60 in port5 ($R \in [0.003856, 0.003971]$, 44 of them hitting the iteration limit). All lie next to R_{max} , and the reason is geometric rather than numerical: reaching a return close to R_{max} forces almost all the budget into the single best asset, so the feasible set shrinks to a very thin corner of the simplex, and at R_{max} exactly it is a single point with no strictly feasible portfolio. The problem is degenerate: at R_{max} only one portfolio is feasible, and the price of return dV/dR becomes huge. OSQP builds that price incrementally, in steps of size ρ , so it needs tens of thousands of iterations to get there. With the default `max_iter = 4000` the damage is much larger (121, 82 and 125 lost points in port2, port4 and port5), and some feasible problems are even reported as *primal infeasible*.

6 Conclusions

The basic Markowitz model is a strictly convex QP, and an open-source first-order solver reproduces the OR-Library reference frontiers to their published precision, provided the tolerance is tightened to 10^{-6} and polishing is enabled. Exploiting the parametric structure — one factorisation plus warm starts — brings each of the 10 000 QPs down to milliseconds. The weak spot is the maximum-return end of the frontier, which is degenerate and where a first-order method is not the right tool: solver choice depends on problem geometry, not only on size.

Code availability. All computations are reproducible from the notebook `markowitz_osqp.ipynb`, delivered separately: open it with the project kernel and *Run All* (≈ 6 min).

References

- [1] J. E. Beasley. Portfolio optimisation: models and solution approaches. *INFORMS TutORials in OR*, 201–221, 2013.
- [2] J. E. Beasley. OR-Library. *J. Oper. Res. Soc.*, 41(11):1069–1072, 1990. Data: people.brunel.ac.uk/~mastjjb/jeb/orlib/portinfo.html
- [3] OSQP Developer Team. *OSQP documentation: Python interface*, 2025. <https://osqp.org/docs/interfaces/python.html>